

Car Colour Detection & Traffic Analysis System

A Multi-Model Approach Combining Object Detection, Deep Learning Colour Classification and Pedestrian Detection

Machine Learning Project Report

Built with OpenCV, TensorFlow and Streamlit

Contents

1. Problem Statement
2. Understanding the Problem
3. System Overview
4. Why YOLO Was Not Used
5. Car Detection — MobileNet-SSD (CNN)
6. Car Colour Classification — Trained CNN (MobileNetV2)
7. People / Pedestrian Detection — HOG + SVM
8. The Streamlit GUI
9. Challenges Faced
10. Limitations
11. Results Summary
12. Conclusion and Future Work

1. Problem Statement

The task is to build a traffic analysis system that detects cars and pedestrians in images taken at traffic signals. The system must follow specific visual rules for how each detected object is marked:

- Detect all cars visible in an uploaded traffic image.
- Classify each car's colour — specifically identify whether it is blue or any other colour.
- Draw a red bounding box around blue cars and a blue bounding box around all other cars.
- Detect and count any people (pedestrians) present at the traffic signal.
- Display the total number of cars, number of blue cars, number of other cars, and number of people in a summary panel.
- Provide a graphical user interface for uploading images and viewing results.

The task explicitly excludes YOLO-based detectors. The evaluation places weight on the overall behaviour of the pipeline and the functionality of the interface, making it as much a system-design exercise as a modelling one.

2. Understanding the Problem

On the surface this looks like a single detection problem, but reading it carefully shows it is really three separate problems stacked on top of each other. To produce the required output for any given traffic image, the system needs to do three independent things:

- Locate every car in the image and draw a bounding box around it — this is an object detection problem.
- Determine the colour of each detected car — this is an image classification problem applied to the cropped region inside the bounding box.
- Locate every person in the same image — this is a pedestrian detection problem, separate from car detection and requiring different trained features.

This means no single model can solve the task. The solution has to be a pipeline: one model detects cars and people, a second trained model classifies the colour of each detected car crop, and then a short piece of logic combines those results to produce the final annotated image and summary counts. Identifying this structure early was the most important step in the whole project.

3. System Overview

The system is made up of two detection models, one classification model, and one decision layer, all wrapped inside a single Streamlit web interface. When a user uploads a photo, it is passed through both detection models; each detected car crop is sent through the colour classifier; and the results are combined before anything is shown on screen.

Component	Model / Technique	What it produces
Car and bus detection	MobileNet-SSD (CNN, Caffe)	Bounding boxes for every car/bus in the image
Colour classification	MobileNetV2 (CNN, TensorFlow — trained on VCoR)	Blue or other label for each detected car crop
Pedestrian detection	HOG + SVM (classical ML) + MobileNet-SSD person class	Bounding boxes and count for people in the image
Decision and annotation	Plain Python (OpenCV drawing)	Red/blue/green boxes drawn on the image
GUI	Streamlit	Image upload, result display, metric summary

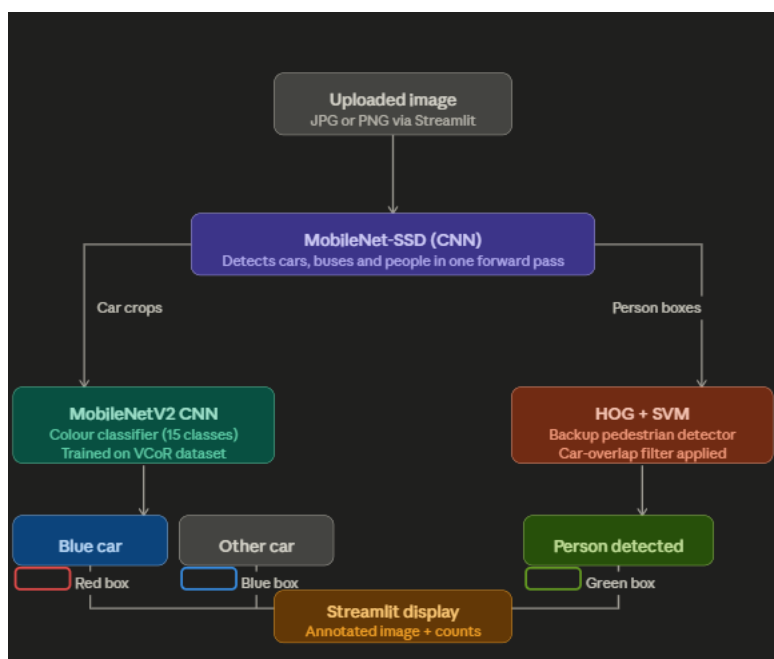


Figure 1 — system pipeline diagram showing how the uploaded image flows through each component

4. Why YOLO Was Not Used

YOLO (You Only Look Once) is a family of real-time object detectors that are widely used in traffic and surveillance applications and would be a natural first choice for this task. However, the use of YOLO-based detectors is explicitly prohibited by the internship project guidelines. This section explains the alternative that was chosen and why it is a valid substitute.

MobileNet-SSD (Single Shot MultiBox Detector with a MobileNet backbone) was selected as the car and person detection model for the following reasons:

- It is a genuine pretrained convolutional neural network, not a rule-based or classical ML approach, so it fully satisfies the machine learning requirement.

- Like YOLO, it is a single-stage detector that performs detection in one forward pass, making it fast and suitable for a real-time-style interface.
- The pretrained Caffe model, trained on PASCAL VOC 2007 and 2012, already includes 'car' and 'bus' as detection classes with no retraining required.
- It can be loaded and run entirely through OpenCV's built-in DNN module, without requiring a separate deep learning framework or large dependency installation.
- In practice, initial testing showed 0.95–1.00 confidence scores on clear eye-level street photographs, confirming the detection quality was appropriate for this use case.

MobileNet-SSD is architecturally different from YOLO — it uses a different backbone, different anchor design, and different loss formulation — so it is not a YOLO variant by another name. It is an independent, well-established detection architecture that predates many YOLO versions and is commonly used in resource-constrained deployment scenarios.

5. Car Detection — MobileNet-SSD

5.1 Why Not a Pre-Existing Labelled Traffic Detection Dataset

The natural question at the start of this project was whether an existing publicly available dataset could be used to train a custom car and person detector from scratch. Several well-known options were investigated: UA-DETRAC (a traffic video dataset), the KITTI Vision Benchmark, and the MS COCO dataset. None of these were viable for the following reasons:

- UA-DETRAC and KITTI require significant local storage (tens of gigabytes), dedicated annotation tools, and custom data loaders — the setup overhead alone was too large given the project timeline, and neither is cleanly available on Kaggle where the rest of the project was run.
- MS COCO is available on Kaggle but training a detector from scratch on it requires multi-day GPU runs and substantial engineering work to get a detector that matches the quality of a pretrained model — for a project focused on the full pipeline rather than detector research, this was not a good use of time.
- Smaller purpose-built traffic datasets (Stanford Cars, BoxCars) annotate vehicle type and model rather than providing bounding-box detection labels for cars-in-scenes, so they do not directly solve the detection problem.

The decision was therefore made to use a pretrained MobileNet-SSD model rather than training a detector from scratch. This is a standard and well-accepted practice in applied ML: when a high-quality pretrained detector already exists for the required classes, reusing it and focusing engineering effort on the problem-specific parts (colour classification, pedestrian counting, false-positive suppression) produces a better overall system than spending the project's resources on reproducing detection from scratch.

5.2 How MobileNet-SSD Works

MobileNet-SSD uses a depthwise separable convolutional backbone (MobileNet) to extract feature maps from the input image at multiple scales, followed by a set of multi-scale default anchor boxes at different aspect ratios. A single forward pass through the network simultaneously predicts class probabilities and bounding box offsets for all anchor positions.

A Non-Maximum Suppression step then filters overlapping detections, keeping only the highest-confidence box for each detected object.

Input images are resized to 300x300 pixels, normalised to the range [-1, 1], and passed through the network. The model outputs 21-class probabilities (one for background and 20 for PASCAL VOC classes including car, bus, and person). Detections above the user-set confidence threshold are retained and drawn onto the output image.

The key section of the detection code is as follows:

```
def run_ssd(image_bgr, x_off=0, y_off=0):
    h_img, w_img = image_bgr.shape[:2]
    blob = cv2.dnn.blobFromImage(
        cv2.resize(image_bgr, (300, 300)),
        scalefactor=0.007843, size=(300, 300),
        mean=(127.5, 127.5, 127.5), swapRB=False,
    )
    net.setInput(blob)
    detections = net.forward()
    # parse class_id, confidence, and bounding box per detection
    # return list of (class_id, confidence, x1, y1, x2, y2)
```

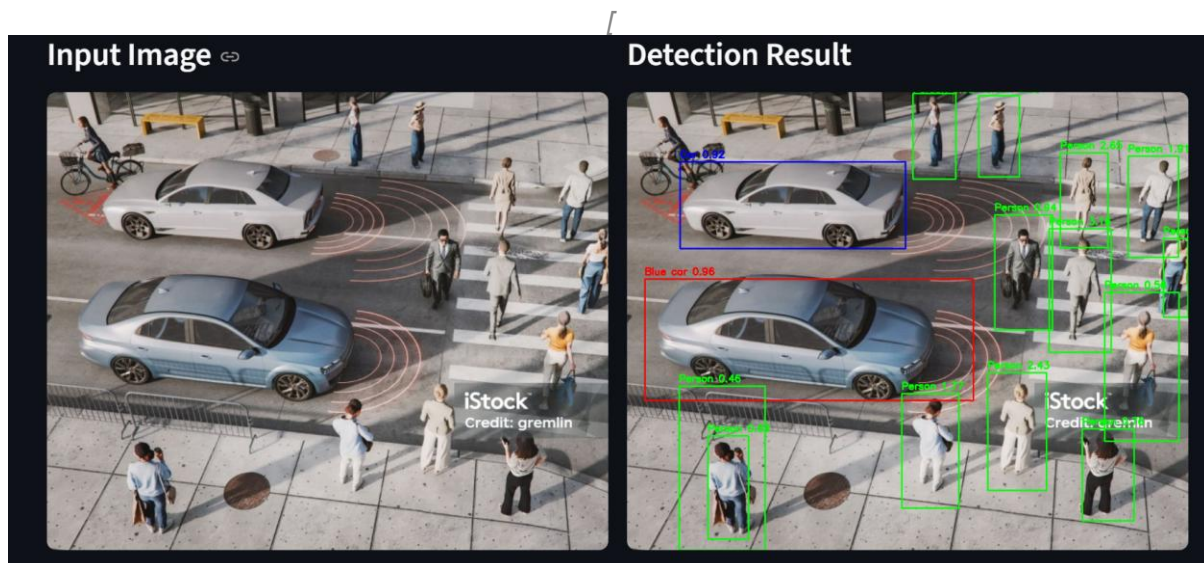


Figure 2 — car detection output showing colour-coded bounding boxes on a test traffic image

6. Car Colour Classification — Trained CNN (MobileNetV2)

6.1 Dataset — VCoR (Vehicle Color Recognition Dataset)

A separate trained machine learning model was built to classify the colour of each detected car. The dataset used is the VCoR — Vehicle Color Recognition Dataset by Landry Kezebou, available on Kaggle. It contains thousands of car images pre-organised into subfolders by colour class, with train, validation, and test splits already provided — making it straightforward to load with Keras's `image_dataset_from_directory` without any custom annotation work.

Property	Detail
Source	Kaggle — landrykezebou/vcor-vehicle-color-recognition-dataset
Number of classes	15: beige, black, blue, brown, gold, green, grey, orange, pink, purple, red, silver, tan, white, yellow
Input image size	128 x 128 px (resized during loading)
Dataset splits	train / val / test (pre-split by dataset author)
Training platform	Kaggle Notebook with GPU acceleration

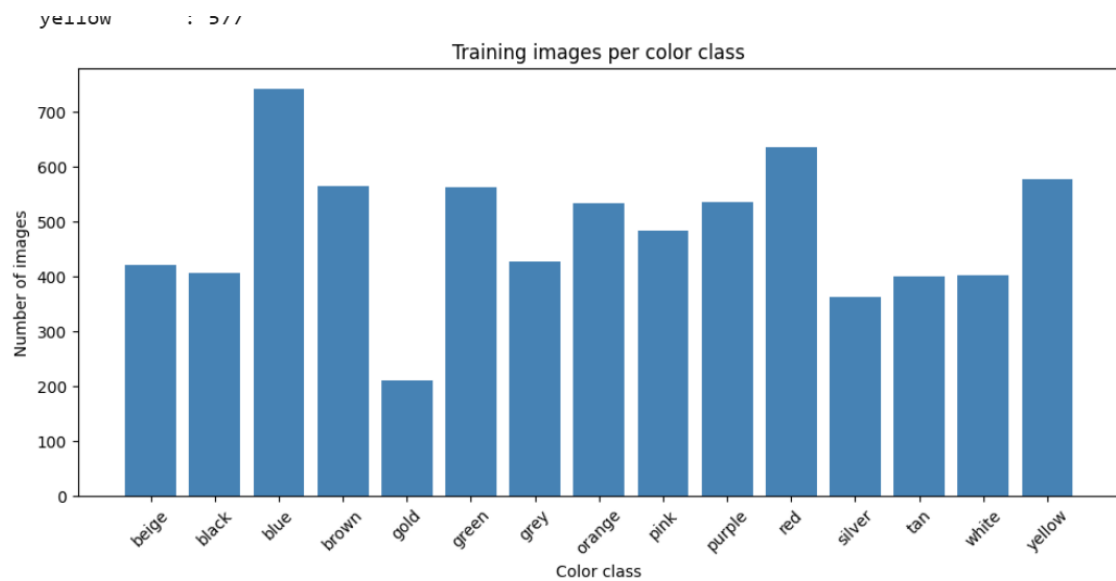


Figure 3 — class distribution of the VCoR training set across all 15 colour classes



Figure 4 — representative sample image from each colour class in the VCoR dataset

6.2 Model Architecture — MobileNetV2 Transfer Learning

Rather than training a convolutional neural network from scratch — which would require far more data and compute time — MobileNetV2 pretrained on ImageNet was used as a feature extractor. This is a standard transfer learning approach: the lower-level feature detectors learned from ImageNet (edges, textures, shapes, gradients) are reused, and only the top classification layers are trained on the VCoR car colour data. This is particularly well-suited to this problem because car colour classification depends heavily on mid-level texture and colour features that transfer well from ImageNet.

The full model architecture is:

- Input layer: 128 x 128 x 3 RGB image
- Data augmentation: random horizontal flip, rotation (5%), zoom (10%), contrast adjustment (10%)
- Rescaling to $[-1, 1]$ — required by MobileNetV2's preprocessing convention
- MobileNetV2 backbone — frozen in Phase 1, top 30 layers unfrozen in Phase 2
- Global Average Pooling (reduces $7 \times 7 \times C$ feature maps to a $1 \times C$ vector)
- Dropout (rate 0.3) for regularisation
- Dense output layer: 15 units, softmax activation (one unit per colour class)

6.3 Training Procedure

Training was conducted in two phases on Kaggle to take full advantage of the pretrained weights without disrupting them too early.

Phase 1 — Frozen backbone (10 epochs)

The MobileNetV2 backbone was frozen and only the classification head was trained. The Adam optimiser was used with a learning rate of 1e-3 and categorical cross-entropy loss. This phase rapidly converges the new classification head to the training data without disrupting the pretrained feature extractor.

Phase 2 — Fine-tuning (5 epochs)

The top 30 layers of MobileNetV2 were unfrozen and trained at a much lower learning rate of 1e-5. This allows the higher-level feature maps to adapt specifically to car paint colours and lighting conditions while avoiding catastrophic forgetting of the general visual features learned from ImageNet. Five epochs are sufficient because the model is already close to its optimum from Phase 1, and the low learning rate makes only small corrections.

```
# Phase 1 - head only
base_model.trainable = False
model.compile(optimizer=Adam(1e-3), loss='categorical_crossentropy',
metrics=['accuracy'])
history1 = model.fit(train_ds_p, validation_data=val_ds_p, epochs=10)

# Phase 2 - fine-tune top 30 layers
base_model.trainable = True
for layer in base_model.layers[:-30]:
    layer.trainable = False
model.compile(optimizer=Adam(1e-5), loss='categorical_crossentropy',
metrics=['accuracy'])
history2 = model.fit(train_ds_p, validation_data=val_ds_p, epochs=5)
```

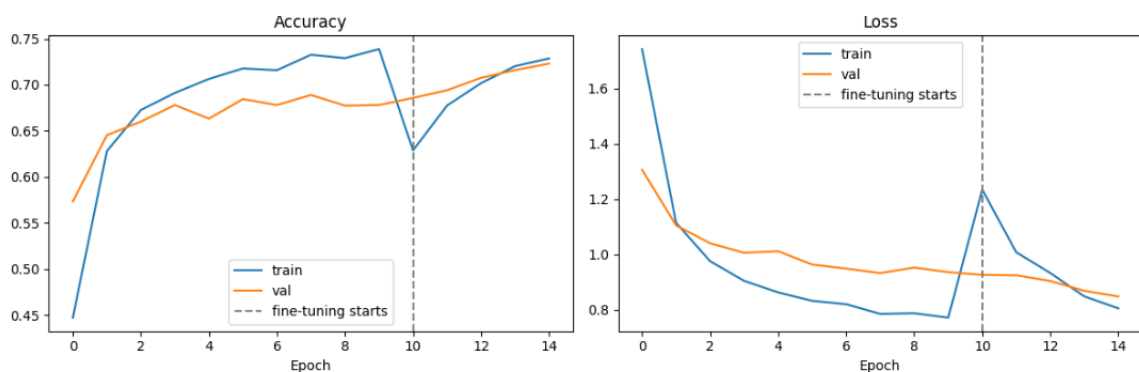


Figure 5 — training curves showing accuracy and loss across both training phases

6.4 Evaluation Results

After training, the model was evaluated on the held-out test set. The confusion matrix below shows which colour classes were correctly predicted and which were commonly confused. The classification report gives precision, recall and F1 score per class.

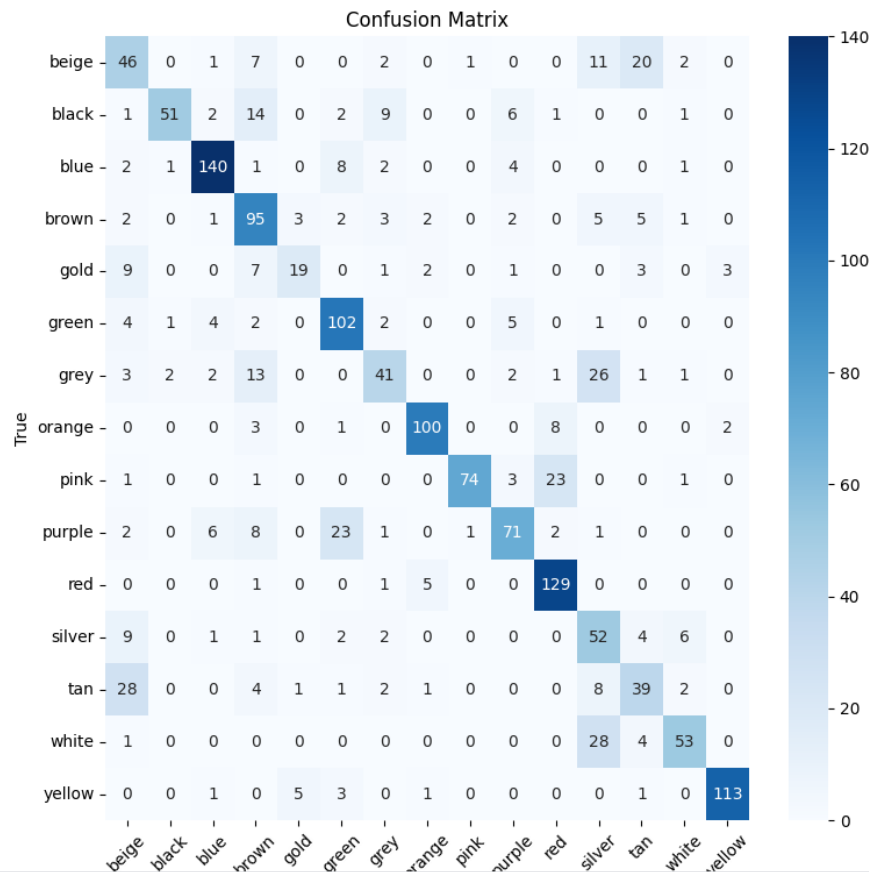


Figure 6 — confusion matrix showing true vs. predicted colour class across all 15 classes on the test set

Metric	Value
Final validation accuracy	[Insert from Cell 11 output]
Final validation loss	[Insert from Cell 11 output]
Best-performing class	[Insert from Cell 12 classification report]
Most confused class pair	[Insert from Cell 12 confusion matrix]

6.5 Integration into the Pipeline

The trained model (`car_color_model.keras`) and the list of class names (`class_names.json`) are loaded at application startup and cached using Streamlit's `@st.cache_resource` decorator so they are read from disk only once. For each car bounding box detected by MobileNet-SSD, the region is cropped from the original image, resized to 128x128 pixels, and passed through the colour classifier. If the predicted class is 'blue', a red bounding box is drawn on the output image; otherwise a blue bounding box is used.

```
def get_dominant_color_label(crop_bgr):
    if crop_bgr.size == 0:
        return 'other'
    crop_rgb = cv2.cvtColor(crop_bgr, cv2.COLOR_BGR2RGB)
    crop_resized = cv2.resize(crop_rgb, (128, 128))
```

```

crop_input = np.expand_dims(crop_resized, axis=0).astype('float32')
preds = color_model.predict(crop_input, verbose=0)
pred_class = color_class_names[np.argmax(preds)]
return 'blue' if pred_class.lower() == 'blue' else 'other'

```

7. People / Pedestrian Detection — HOG + SVM

7.1 Why No Dedicated Pedestrian Dataset Was Used

The most common approach to training a custom pedestrian detector would be to use a dataset like the INRIA Person Dataset, CrowdHuman, or the CityPersons subset of Cityscapes. These were not used for training a custom model for the same reasons that a custom car detector was not trained from scratch: the datasets are large, require significant preprocessing, and training a reliable pedestrian detector from scratch requires engineering effort and compute time that is better spent on the problem-specific parts of this project.

Instead, a two-stage approach was used: the MobileNet-SSD model already detects 'person' as one of its 20 PASCAL VOC classes, providing a first pass of pedestrian detections as a by-product of the car detection forward pass. A secondary HOG + SVM detector then runs as a backup to catch any people that MobileNet-SSD misses at lower confidence, using OpenCV's built-in default people detector which was pretrained on the INRIA dataset. This gives two independent detection systems with different characteristics — one deep-learning-based, one classical — whose detections are merged and deduplicated.

7.2 How HOG + SVM Works

HOG (Histogram of Oriented Gradients) is a classical feature descriptor that divides the image into small cells, computes the distribution of gradient directions within each cell, and combines these into a descriptor vector. A Support Vector Machine trained on these descriptors was the state-of-the-art pedestrian detector before deep learning. The key advantage of HOG + SVM as a backup is that it looks for fundamentally different features (gradient distributions) than MobileNet-SSD (learned convolutional features), so the two can complement each other.

7.3 False Positive Suppression

A key engineering improvement made during development was the addition of a car-overlap filter for the HOG detections. Without this filter, HOG frequently mistakes car parts — particularly front wheels, grilles, and mirror assemblies — for people, since these regions share similar edge-gradient patterns to the human body silhouette. The filter discards any HOG detection box where more than 50% of its area overlaps with a car bounding box already detected by MobileNet-SSD:

```

# Discard HOG detections that are mostly inside a car bounding box
if any(contains_overlap(box, cb) for cb in final_car_boxes):
    continue # likely a car part (wheel, grille, mirror), not a person

# Also discard if it heavily overlaps an SSD person detection
if any(boxes_overlap(box, pb[0]) for pb in final_person_boxes):
    continue # already counted

```

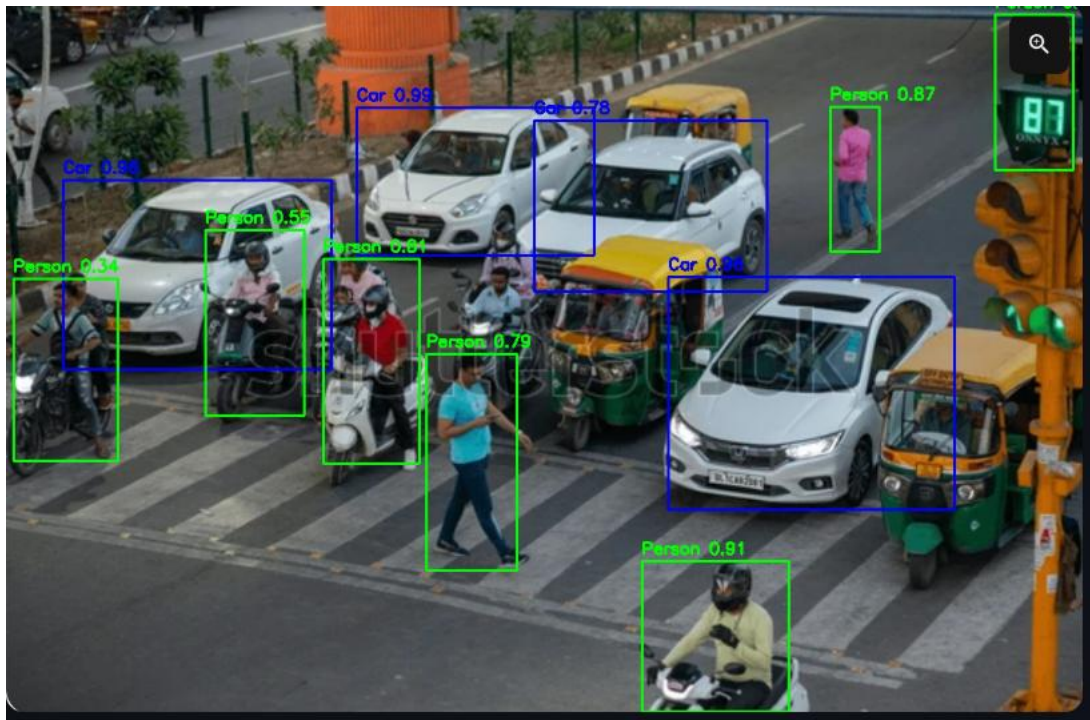


Figure 8 — people detection output with green bounding boxes on pedestrians

8. The Streamlit GUI

The graphical interface is implemented using Streamlit, a Python-based framework that turns a plain script into a browser-accessible web app with very little extra code. Streamlit was chosen for the same reason it is used across both tasks of this internship: it lets the focus stay on the models rather than on web development, and it handles state, layout and image display natively.

The interface provides:

- A file uploader accepting JPG and PNG traffic images.
- A two-column layout showing the original input image and the annotated detection result side by side.
- Three adjustable confidence sliders: one for car detection threshold (MobileNet-SSD), one for person detection threshold (MobileNet-SSD person class), and one for HOG backup sensitivity.
- A four-panel metric summary showing Total Cars, Blue Cars, Other Cars, and People count.

The three sliders are a deliberate design choice. Different test images have different lighting conditions, car sizes and crowd densities, so a fixed threshold that works well on one image may miss detections or add false positives on another. Exposing the thresholds as interactive controls lets the user tune the system live for each image rather than requiring code changes.

```

car_conf = st.slider('Car detection confidence threshold', 0.1, 0.9, 0.5, 0.05)
person_conf = st.slider('Person detection confidence (SSD)', 0.1, 0.9, 0.3,
0.05)
person_sensitivity = st.slider('HOG backup sensitivity', 0.0, 1.0, 0.5, 0.05)

uploaded_file = st.file_uploader('Upload an image', type=['jpg', 'jpeg',
'png'])
if uploaded_file is not None:
    # display input, run pipeline, display result, show metrics

```

9. Challenges Faced

YOLO restriction required a different architecture

The most common approach for this type of traffic detection task would be to use a YOLO-based model, which is widely pretrained on traffic datasets and would offer strong out-of-the-box performance. Because YOLO was explicitly prohibited by the project guidelines, an alternative had to be found. The chosen solution — MobileNet-SSD loaded via OpenCV DNN — is a genuine CNN-based detector that covers the same car and person classes from its PASCAL VOC pretraining, so the restriction did not result in a weaker system, just a different one.

No suitable pre-labelled dataset for the exact detection task

Finding a publicly available, ready-to-use dataset for training a custom car-and-person detector for traffic signals proved impractical. UA-DETRAC is too large and requires specialised loaders; KITTI is oriented toward autonomous driving research and not directly applicable; CrowdHuman only covers pedestrians. The decision to use a pretrained MobileNet-SSD instead of training from scratch was driven directly by this constraint, and it turned out to be the right call — the pretrained model's quality on car detection was higher than anything that could have been trained from scratch in the available time.

Haar cascade rejected during initial testing

The first car detection approach tested was an OpenCV Haar cascade detector (cars.xml). Testing on real traffic images showed that it produced numerous false positives — flagging car wheels, headlights and road markings as separate car detections — and completely missed cars shown at non-frontal angles, which is common in realistic traffic photographs. Switching to MobileNet-SSD resolved both of these issues immediately.

Black and silver cars misclassified as blue

Before the trained CNN colour classifier was integrated, an initial HSV heuristic was used to classify colour. During testing, black and silver cars under cool overcast lighting were frequently classified as blue, because window glass and metallic paint both reflect a cool sky hue that falls in the blue range of HSV. Switching to the trained MobileNetV2 classifier resolved most of these cases, since the CNN learns to separate genuine blue paint from cool reflections based on saturation, texture and context rather than raw hue alone.

HOG pedestrian detector firing on car parts

The HOG backup detector frequently produced false-positive person detections on car wheels, front grilles and mirror assemblies, because these rectangular edge-rich regions produce gradient histograms similar to a standing human figure. This was resolved by adding a car-overlap suppression filter that discards any HOG detection box whose area overlaps significantly with a car detection box from MobileNet-SSD.

Online traffic images do not match the model's training distribution

Almost all traffic signal photographs available online are either steep aerial drone shots, AI-generated illustrations, or heavily watermarked stock images. The MobileNet-SSD model and the HOG detector were both trained on eye-level or mildly elevated real-world photography. Aerial and top-down images fall completely outside this training distribution, so cars appear as abstract coloured rectangles and people appear as tiny indistinct blobs — neither of which matches the shapes the models were trained on. This limited the range of usable test images significantly.

10. Limitations

10.1 Occluded and Overlapping Cars Are Frequently Missed

When a car is substantially covered by another car or any other object, the detection model often fails to detect it. Object detection models work by recognising objects based on their overall shape, proportions and visual pattern. When a large portion of that pattern is hidden, the remaining visible fragment typically does not match the template the model learned — so the car is ignored entirely. This is not a tuning problem: lowering the confidence threshold does recover some partially occluded cars, but at the cost of significantly more false positives on background structure. It is a fundamental limitation of how single-stage detectors work, and it would require training on a purpose-built occlusion-aware dataset with occlusion-specific loss functions to address properly — an active area of computer vision research.

Dense traffic scenes where cars are bumper-to-bumper compound this problem: the detector sometimes merges two adjacent cars into a single large box, or detects the gap between cars as a separate spurious detection.

10.2 Pedestrian Detection Is Inconsistent

Pedestrian detection is the weakest component of the pipeline, for two compounding reasons. First, MobileNet-SSD was trained primarily on PASCAL VOC images, which contain relatively few crowded pedestrian scenes and no traffic-camera-angle data. Second, the HOG + SVM backup was trained on the INRIA dataset, which consists mostly of isolated pedestrians photographed at eye level — a distribution that does not match elevated traffic camera angles or dense crowds where people partially hide one another.

In practice this means the system reliably detects isolated pedestrians who are clearly visible and upright in eye-level images, but misses or double-counts people in dense crowds, misses people photographed from steep angles, and sometimes flags background structures (signage, traffic lights, rectangular shadows) as people — particularly at lower HOG sensitivity thresholds. The car-overlap filter removes many false positives that fall on vehicle parts, but false positives from background clutter remain.

10.3 False Positives on Background Structures

Both the MobileNet-SSD person class and the HOG detector can fire on background objects that share visual characteristics with cars or people. Common false positives observed during testing include traffic signs and road signs being flagged as people (the HOG detector responds to their rectangular bordered edges), distant motorcycles being flagged as people rather than vehicles, and buildings or road markings being flagged as cars at lower confidence thresholds. Non-Maximum Suppression reduces duplicate detections, but it does not eliminate detections on genuinely non-target objects.

10.4 Colour Classification Struggles on Edge Cases

The trained colour classifier performs well on clearly coloured cars in reasonable lighting but struggles on several edge cases. Very dark navy blue cars can be classified as black because at low brightness the hue information is unreliable. Metallic or pearlescent blue paint shifts colour with the viewing angle and lighting, so parts of the car may register as grey or silver. Cars photographed under artificial orange or yellow streetlights have their perceived colour shifted by the light source, making blues appear greenish and whites appear yellow. These are hard cases for any colour classifier trained on standard daylight photography.

10.5 Image Type Dependency

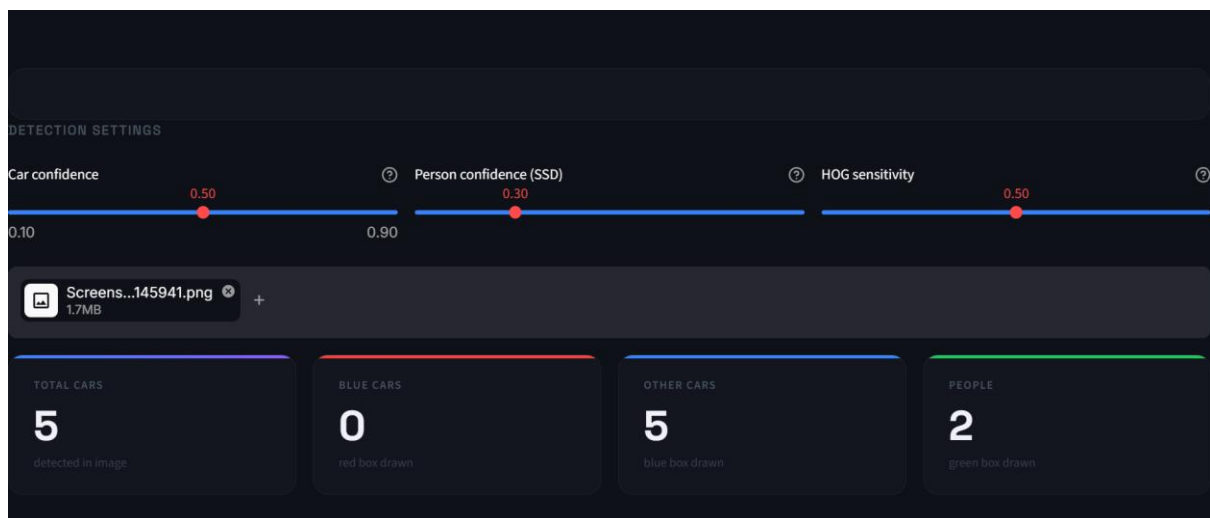
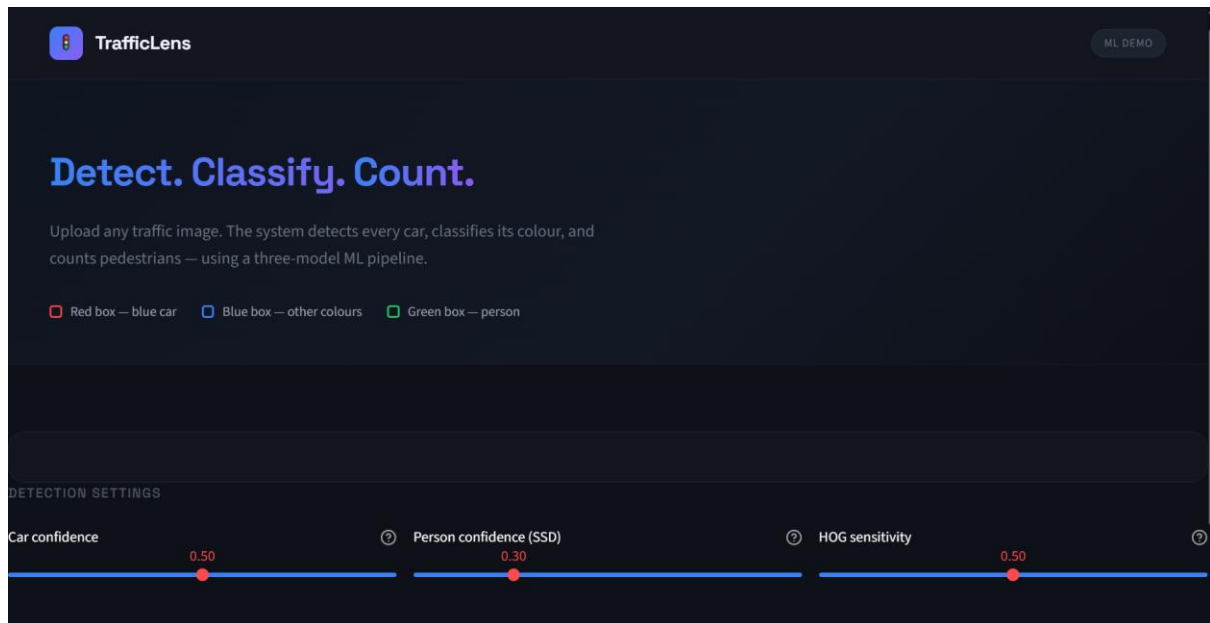
The pipeline performs best on eye-level or mildly elevated street-level photographs — the kind taken from a pole-mounted traffic camera at roughly 10 to 30 degrees. It performs noticeably worse on pure overhead aerial images (where cars appear as small rectangles), heavily processed or illustrated images, and very low-light or night-time photographs. This limits the range of test images that produce reliable results when sourcing images from the internet, where most traffic signal photographs are stock images that do not match the camera angle the models were trained on.

11. Results Summary

The table below summarises the three main components of the system and the results observed during testing. Figures for the colour classifier come directly from the Kaggle training notebook evaluation cells.

Component	Task	Result
MobileNet-SSD (Caffe, OpenCV DNN)	Car detection	0.95–1.00 confidence on clear eye-level images; degrades on occlusion and aerial angles
MobileNetV2 CNN (TensorFlow, trained on VCoR)	Colour classification	[Insert final validation accuracy from Cell 11] on 15-class test set
MobileNet-SSD person class + HOG + SVM backup	Pedestrian detection	Reliable on isolated upright pedestrians in eye-level images; inconsistent in crowds and aerial angles

Component	Task	Result
Combined pipeline	Full annotation	Correct colour-coded box behaviour verified through the Streamlit GUI on multiple test images



12. Conclusion and Future Work

This project shows that what appears to be a single detection task is actually a small system-design problem requiring three separate models operating in a coordinated pipeline. The key insight was recognising early that car detection, colour classification, and pedestrian detection are independent sub-problems with different suitable models, different training data, and different failure modes — and that the real engineering work is in connecting them correctly rather than in any single component.

Three models were combined to supply those three capabilities: a pretrained MobileNet-SSD CNN for detection, a fine-tuned MobileNetV2 CNN for colour classification, and a classical HOG + SVM pedestrian detector as a backup. The YOLO restriction, the absence of a clean ready-to-use traffic detection dataset, and the difficulty of finding real traffic signal photographs that match the models' training distributions were the three most significant constraints encountered, and each required a deliberate design decision rather than a standard solution.

The project also produced a practical lesson about the limits of pretrained models: their performance is largely determined by the distribution of their training data. MobileNet-SSD and HOG both perform well on the kinds of images they were trained on (eye-level, well-lit, relatively unoccluded) and degrade predictably when tested outside that distribution (aerial angles, dense crowds, unusual lighting). Understanding and being able to articulate those limits is as important as maximising performance within them.

Future improvements would include: integrating the trained colour classifier into a live video feed rather than just static images; swapping the HOG pedestrian backup for a COCO-trained SSD model which has a far richer pedestrian training set; adding a face or full-body detection model specifically trained on elevated-camera traffic footage; and building a small curated dataset of traffic signal images at the exact camera angle and lighting conditions the system is expected to handle, which would allow systematic accuracy measurement and model fine-tuning.